Day 21, HTTP Request Library

Think of HTTP like ordering at a restaurant. You (the client) place an order, the kitchen (the server) prepares it, and a waiter brings back a response.

1. URLs and HTTP requests

A **URL** is the address of something on the internet.

https://api.example.com/users?id=5

- https:// is the protocol (the "language" used, and the s means encrypted)
- api.example.com is the server's address
- /users is the specific resource you want
- ?id=5 is extra details (a query parameter)

An **HTTP request** is your message to that address. The most common types:

Method	Purpose
GET	Read or fetch data
POST	Send new data
PUT / PATCH	Update data
DELETE	Remove data

2. GET request using requests

requests is a Python library that makes sending these messages easy.

python

```
import requests


response = requests.get("https://jsonplaceholder.typicode.com/posts/1")


print(response.status_code)  # 200
print(response.json())       # data as a Python dictionary
```

response.json() is the one you'll use most, since most APIs return JSON, which Python turns into a dictionary you can work with:

python

```
data = response.json()
print(data["title"])
```

3. HTTP status codes

Every response includes a number telling you what happened. Look at the first digit:

- **2xx: Success.** 200 OK
- **3xx: Redirect.** The thing moved somewhere else.
- **4xx: Your mistake.** 404 Not Found (wrong URL), 401 Unauthorized (not logged in), 403 Forbidden (not allowed)
- **5xx: Server's mistake.** 500 Internal Server Error

python

```
if response.status_code == 200:
    print("Success!")
elif response.status_code == 404:
    print("Not found")
```

Or use the shortcut that raises an error automatically on 4xx/5xx:

python

```
response.raise_for_status()
```

4. Request timeout

A **timeout** is the maximum time you'll wait for the server to reply.

Why it matters: by default, requests waits *forever*. If the server is down or very slow, your program freezes. In the restaurant analogy, you'd sit there for hours instead of saying "if my food isn't here in 10 minutes, I'm leaving."

python

```
try:
    response = requests.get("https://httpbin.org/delay/5", timeout=2)
except requests.exceptions.Timeout:
    print("Server took too long!")
```

Here the server waits 5 seconds to respond, but we only wait 2, so the Timeout exception fires and we handle it gracefully instead of hanging.

Rule of thumb: always set a timeout (usually 5 to 10 seconds).

Putting it all together

python

```
import requests
```

```
try:
    r = requests.get("https://jsonplaceholder.typicode.com/posts/1", timeout=5)
    r.raise_for_status()
    print(r.json()["title"])
except requests.exceptions.Timeout:
    print("Request timed out")
except requests.exceptions.HTTPError as e:
    print("Bad status code:", e)
except requests.exceptions.ConnectionError:
    print("Could not connect")
```

This one snippet uses all four topics: a URL, a GET request, status code checking, and a timeout.

If any part is still unclear, tell me which one and I'll explain it another way, or give you practice exercises.

now make one program fetch data from json fake api

Here's a complete program that fetches data from JSONPlaceholder (a free fake API) and uses everything from Day 21.

python

```
import requests

# 1. Fetch one post
response = requests.get("https://jsonplaceholder.typicode.com/posts/1", timeout=5)

# 2. Check the status code
print("Status:", response.status_code)

# 3. Use the data if the request worked
if response.status_code == 200:
    data = response.json()
    print("Title:", data["title"])
    print("Body:", data["body"])
else:
    print("Something went wrong")
```

Try it yourself

1. Run it: python fetch_api.py
2. Change fetch_post(1) to a different ID.
3. Add a function that fetches comments from /posts/1/comments.
4. Print the user's name from /users/1.